Modelling and analysis of a cyber-physical system (now with monads!!)

Cláudia Faria (PG60240)

Universidade do Minho, Braga, Portugal
PG60240@alunos.uminho.pt

June 10, 2026

1 Introduction

This project was done for the Cyber-Physical Programming class in the Formal Methods of Programming specialization of the Master's in Informatics Engineering at the University of Minho.

The goal of this project is to model systems where actions take time, choices are non-deterministic, and outcomes are probabilistic. Using monads, the system supports property verification and the analysis of different execution paths.

2 Problem

In the middle of the night four adventurers encounter a rope-bridge that spans a deep ravine. For safety reasons, no more than two people cross the bridge at the same time and a flashlight needs to be carried in every crossing. There is only one flashlight. The four adventurers are not equally skilled: crossing the bridge takes them one, two, five, and ten minutes respectively. A pair of adventurers crosses the bridge in an amount of time equal to that of the slowest of the two.

One adventurer claims that they cannot be all on the other side in less than 19 minutes. Another one disagrees and claims that it can be done in 17 minutes.

- **First stage:** Verifying the claims of the adventurers.
- **Second stage:** Implementing probabilistic crossing times.
- **Third stage:** Adding new functionalities.

3 Monads

A Monad is a type constructor essential to handle side effects, failures, or other unexpected situations in a pure, composable way. Monads ListDur and DistDur were provided for this assignment.

We will use these Monads to lift values to monadic context and compose monadic computations into a single pipeline.

3.1 Non-determinism combined with durations

The Duration monad tracks how long each action takes by accumulating time as computations are sequenced. The ListDur monad layers non-determinism on top of duration.

```
data ListDur a = LD [Duration a] deriving Show

remLD :: ListDur a -> [Duration a]
remLD (LD x) = x

instance Functor ListDur where
   fmap f = let f' = (fmap f) in
     LD . (map f') . remLD

instance Applicative ListDur where
   pure x = LD [Duration (0,x)]
   l1 <*> l2 = LD $ do x <- remLD l1
                       y <- remLD l2
                       return $ do f <- x; a <- y; return (f a)

instance Monad ListDur where
   return = pure
   l >>= k = LD $ do x <- remLD l
                     g x where
                       g(Duration (i,x)) = let u = (remLD (k x))
                          in map (\(Duration (i',x)) -> Duration (i +
                              i', x)) u
```

3.2 Probabilistic behaviour combined with durations

Then, the DistDur monad assigns probabilities to each outcome.

```
data DistDur a = DD (Dist (Duration a)) deriving Show

remDD :: DistDur a -> Dist (Duration a)
remDD (DD x) = x

instance Functor DistDur where
        fmap f = DD . (fmap (fmap f)) . remDD

instance Applicative DistDur where
        pure x = DD (return $ return x)
        d1 <*> d2 = DD $ do x <- remDD d1
```

```
                              y <- remDD d2
                              return $ do f <- x; a <- y; return (f a)

instance Monad DistDur where
        return = pure
        d >>= k = DD $ do x <- remDD d
                          g x where
                          g(Duration (i,x)) = let u = (remDD (k x))
                                in fmap (\(Duration (i',x)) ->
                                    Duration (i + i', x)) u
```

4 Verifying Adventurer's Solution

The first step was to model the system using Haskell. Particularly, by defining the adventurers, the objects of the model (groups of 1 or 2 adventurers crossing the bridge), and states. Each adventurer was defined according to the time it took them to cross the bridge.

```
-- List of adventurers
data Adventurer = P1 | P2 | P5 | P10 deriving (Show,Eq)

-- Adventurers + the lantern
type Objects = Either Adventurer ()

-- State of the game
type State = Objects -> Bool
```

The first state of the game is one where all adventurers are on the unsafe side. We must also define what reachable state will be considered as the final one. In this case, it will be when all adventurers have safely made it to the safe side (with the lantern).

```
gInit :: State
gInit = const False

gFinal :: State -> Bool
gFinal s = all (== True) [s (Left P1), s (Left P2), s (Left P5), s (
    Left P10), s (Right ())]
```

The game progresses by changing state. We will define a function that changes an object's state by reversing the Boolean value. Then, we will define a second function that gets a list of objects and threads the function and changes state once per object.

```
-- Changes the state s of the game for a given object o
changeState :: Objects -> State -> State
changeState o s = \x -> if x == o then not (s o) else s x

-- Changes the state of the game for a list of objects
mChangeState :: [Objects] -> State -> State
mChangeState os s = foldr changeState s os
```

4.1 Modelling the system with monads

For a given state of the game, we will calculate all possible moves that the adventurers can make. The following function:

- gets the current position of the lantern;
- filters for the adventurers on the same side as the lantern;
- creates subsets of 1 or 2 available adventurers;
- creates an auxiliary function ToPlay that converts a given group into a ListDur state, with the duration of the maximum time of the selected adventurers and moving their stat;

- unwraps each ListDur, concatenates all lists, and re-wraps, by using non-deterministic choice.

```
allValidPlays :: State -> ListDur State
allValidPlays s = manyChoice validGroups
  where
    lanternSide = s (Right ()) -- what side lantern is on
    available = filter (\a -> s (Left a) == lanternSide) [P1, P2, P5,
        P10] -- adventurers on the same side as the lantern
    soloMoves = map (toPlay . singleton) available -- moves with one
        adventurer
    pairMoves = map (toPlay . pairToList) (makePairs available) --
        moves with two adventurers
    validGroups = soloMoves ++ pairMoves -- all valid moves

    toPlay group =
      let time      = maximum (map getTimeAdv group) -- maximum time of
           2 adventurers
          objects   = map Left group ++ [Right ()] -- objects to move (
              adventurers + lantern)
          newState = mChangeState objects s -- new state
      in  LD [Duration (time, newState)]
```

For a given number n and initial state, we want to calculate all possible n-sequences of moves that the adventurers can make. For this, we will use monadic binding.

```
exec :: Int -> State -> ListDur State
exec 0 s = return s
exec n s = allValidPlays s >>= exec (n - 1)
```

4.2 Verification

To verify if it is possible for all adventurers to be on the other side less or equal to 17 minutes and not exceeding 5 moves:

```
leq17 :: Bool
leq17 = any (\(Duration (t, s)) -> t <= 17 && gFinal s) (remLD (exec 5
    gInit))
leq17check = filter (\(Duration (t,s)) -> t <= 17 && gFinal s) (remLD
    (exec 5 gInit))
```

```
ghci> l17
False
ghci> leq17check
[Duration (17,["True","True","True","True","True"]),Duration
    (17,["True","True","True","True","True"])]
```

To verify if it is possible for all adventurers to be on the other side less or equal to 17 minutes and not exceeding 5 moves:

```
l17 :: Bool
l17 = any (\(Duration (t, s)) -> t < 17 && gFinal s) (remLD (exec 5
    gInit))
```

```
ghci> l17n 7
False
```

Or any specific number of moves:

```
l17n :: Int -> Bool
l17n n = any (\(Duration (t, s)) -> t < 17 && gFinal s) (remLD (exec n
     gInit))
```

4.3 Path visualisation

This functionality extends the first task of verifying the claims of the adventurers. A new function, *printLeq*, allows the user to look for solutions with less or equal to t minutes and n moves, with the first path found being displayed.

```
printLeq :: Int -> Int -> IO ()
printLeq n t = case solutions of
  [] -> putStrLn "No solution found."
  (Duration (_, (path, _))):_ -> printPath path
  where
    solutions = filter (\(Duration (d, (_, s))) -> d <= t && gFinal s)
        (remLD (execWithPath n gInit))
```

```
ghci> printLeq 5 17
Initial: ["False","False","False","False","False"]
Step 1 [+2 min]: ["True","True","False","False","True"]
Step 2 [+1 min]: ["False","True","False","False","False"]
Step 3 [+10 min]: ["False","True","True","True","True"]
Step 4 [+2 min]: ["False","False","True","True","False"]
Step 5 [+2 min]: ["True","True","True","True","True"]
```

This functionality would also be helpful for visualising the traces found for different time restraints.

5 Probabilistic Crossing Times

5.1 Implementing probabilistic crossing times

In this stage, each adventurer's crossing time will now be given by a uniform distribution. If the adventurer's crossing time is x then its probabilistic crossing time will now be given by

$$prob(x) = \frac{1}{3}\left(x - \frac{x}{2}\right) + \frac{1}{3}x + \frac{1}{3}\left(x + \frac{x}{2}\right)$$

This means that each adventurer with time x gets a uniform distribution over three outcomes. Simplified, the outcomes are $x - \frac{x}{2}$, x, $x + \frac{x}{2}$ each with probability $\frac{1}{3}$.

```
prob :: Adventurer -> Dist Int
prob adv = uniform [x `div` 2, x, x + x `div` 2]
  where x = getTimeAdv adv
```

```
ghci> prob P1
1   66.7%
0   33.3%

ghci> prob P2
1   33.3%
2   33.3%
3   33.3%

ghci> prob P5
2   33.3%
5   33.3%
7   33.3%

ghci> prob P10
 5   33.3%
10   33.3%
15   33.3%
```

Similarly to *allValidPlays*, a new function will be implemented to calculate the resulting state based on which adventurers will move next and their probabilistic crossing times.

```
type Move = Either Adventurer (Adventurer, Adventurer)

play :: Move -> State -> DistDur State
play move s = DD (fmap (\t -> Duration (t, newState)) groupTimeDist)
  where
    advs = case move of -- get the adventurer(s) to move
             Left a       -> [a]
             Right (a, b) -> [a, b]
    objects       = map Left advs ++ [Right ()] -- objects to move (
        adventurers + lantern)
    newState      = mChangeState objects s -- new state after moving
        the adventurers
```

```
    groupTimeDist = fmap maximum (mapM prob advs) -- distribution of
        the time taken
```

Then, the function is extended to lists of movements using the monadic bind.

```
plays :: [Move] -> State -> DistDur State
plays [] s = return s
plays (m:ms) s = play m s >>= plays ms
```

5.2 Back to the claims

Previously, we verified that the adventurers can all cross the bridge in 17 minutes. However, in a probabilistic setting, the crossing times are not guaranteed and the distribution may raise the durations at each step and therefore the total duration, making our verifications not viable for this setting.

To account for this, an extra functionality will be introduced to calculate the full distribution for our obtained optimal trace. With the following function, we will re-evaluate the claims of the two adventurers, but this time considering the full distribution.

```
evaluateClaims :: DistDur a -> IO ()
evaluateClaims dd = do
  let dist     = timeDist dd
      pUnder19 = probLessThan 19 dd
      pExact17 = probExact    17 dd
      pUnder17 = probLessThan 17 dd

  putStrLn "Adventurer Claims"
  putStrLn ("P(crossing < 19 min) = " ++ show pUnder19)
  putStrLn ("P(crossing = 17 min) = " ++ show pExact17)
  putStrLn ("P(crossing < 17 min) = " ++ show pUnder17)
```

```
ghci> evaluateClaims (plays optimalMove gInit)
Adventurer Claims
P(crossing < 19 min) = 0.6954757
P(crossing = 17 min) = 5.3498123e-2
P(crossing < 17 min) = 0.6008252
```

It seems that now, it is possible to cross in under 17 minutes as well, with a probability of around 60%! However, our adventurers should not become too optimistic, as there is a 40% probability of this same optimal solution leading to a longer time.

6 Interactive game

The last implemented functionality is a bit more fun: a terminal game loop with the IO monad where the user picks which adventurers to move each turn and the terminal shows the current state after each move, accumulated time, and remaining adventurers.

```
ghci> main
Adventurers Game
P1=1  P2=2  P5=5  P10=10  (minutes)
Optimal: 17 min in 5 moves

-----------------------------------------------------
  Elapsed time : 0 min
  Lantern      : LEFT
  LEFT: P1  P2  P5  P10
 ~~~~~~~~~~~~~~~~~~~~~~ BRIDGE ~~~~~~~~~~~~~~~~~~~~~~
  RIGHT: (empty)
-----------------------------------------------------
Move #1
Adventurers who can cross:
    P1 (1 min)
    P2 (2 min)
    P5 (5 min)
    P10 (10 min)
Who crosses? (e.g. '1' or '2 10'):
```

The game updates the side of the lantern and adventurers every time for easier comprehension and shows the elapsed time.

```
1 2
 > P1 & P2 cross in 2 min.

-----------------------------------------------------
  Elapsed time : 2 min
  Lantern      : RIGHT
  LEFT: P5  P10
 ~~~~~~~~~~~~~~~~~~~~~~ BRIDGE ~~~~~~~~~~~~~~~~~~~~~~
  RIGHT: P1  P2
-----------------------------------------------------
Move #2
Adventurers who can cross:
    P1 (1 min)
    P2 (2 min)
Who crosses? (e.g. '1' or '2 10'):
```

The game ends when all adventurers are safely on the right side and displays the total time and the number of moves taken.

```
1 2
 > P1 & P2 cross in 2 min.
```

```
--------------------------------------------------
  Elapsed time : 17 min
  Lantern      : RIGHT
  LEFT: (empty)
 ~~~~~~~~~~~~~~~~~~~~~~ BRIDGE ~~~~~~~~~~~~~~~~~~~~~~
  RIGHT: P1  P2  P5  P10
--------------------------------------------------
Yay! Everyone crossed!
Total time: 17 min  |  Moves: 5
--------------------------------------------------
```

7 Conclusion

The monadic implementation of a cyber-physical system allows us to model the state and branching in a pure and composable way. Particularly, this system uses monads to model duration, non-determinism, and probability in order to verify properties and compare execution paths of the adventurers' puzzle.

Additionally, extra functionalities such as visualisation functions and an interactive game were implemented in order to enrich the experience for the user and further experiment with monads. Particularly, the IO monad is essential for handling errors and terminal operations.

This project was helpful to learn about programming with monads, particularly the use of monadic binding, $fmap$, and specific monads such as IO and probabilities.

Actions take time, choices branch, and outcomes are uncertain. Non-determinism and probabilistic settings are common in real systems, which rarely behave in an easily predictable way. Therefore, the use of monads is vital to compose, test and reason about these complex systems in a safe manner.